

LAB4-D. Primjer stvaranja i ručnog pokretanja spremnika kroz Docker u Ubuntu na WSL-u

Vremenska ograničenja su za SRSV najbitnija. Ona se ostvaruju kroz prikladnu programsku potporu. Međutim, u složenijim sustavima takva je programska potpora raspodijeljena i prostorno i funkcionalno. U ovoj vježbi se upoznajemo s pomoćnim sustavom za upravljanje spremnicima (kontejnerima, Docker slikama) koji omogućuje pokretanje aplikacija kao spremnika te prati njihov rad i po potrebi stvara više instanci, ponovno pokreće, zamjenjuje aplikacije njihovima novijim inačicama i slično.

Zadatak ove vježbe je prikazati primjenu spremnika za ostvarenje robusnosti

- Ostvariti primjer sustava koji koristi spremnike radi osiguranja neprekinute dostupnosti svojih usluga
- Minimalno što treba ostvariti:
 - a. ostvariti nekoliko usluga (min. 3) - svaka kao zasebna aplikacija pokrenuta kroz mehanizam spremnika, barem kao jedna instanca (a može i više)
 - b. kroz upravitelj spremnika (npr. Docker, yaml datoteku) definirati svojstva svakog spremnika i što napraviti u posebnim slučajevima
 - c. neka usluge međusobno komuniciraju neizravno, putem pouzdane usluge razmjene poruka (npr. RabbitMQ)
 - d. opcija 1.: u neke od usluga (može i samo jednu) namjerno ubaciti grešku, tj. "neočekivani" ispad usluge, a da bi se ispitalo potrebno vrijeme za oporavak (ponovno pokretanje od upravitelja spremnika)
 - e. opcija 2.: pratiti opterećenje sustava, npr. kroz broj poruka u nekom redu, te dinamički dodavati instance neke usluge i micati ih kad više nisu potrebne (na bilo koji način)

U nastavku je u kratkim crticama prikazan primjer stvaranja i ručnog pokretanja spremnika kroz Docker u Ubuntu na WSL-u. Slično bi trebalo raditi i u "običnom Ubuntu" ili čak i kroz druge aplikacije, primjerice kroz [Docker Desktop](#).

Ostvarenje je podijeljeno u korake.

1. Instalirati Docker

```
sudo apt-get update
sudo apt-get upgrade
sudo apt-get install docker.io
```

2. Pokrenuti servis

```
sudo systemctl start docker
```

U WSL-u gornja naredba vjerojatno neće raditi pa pokrenuti ručno:

```
sudo dockerd
```

3. Da bi spremnike mogli pokretati kao korisnik, bez sudo treba korisnika dodati u grupu docker.

```
sudo groupadd docker
sudo gpasswd -a $USER docker
```

Na Ubuntu nakon ovoga napraviti logout/login prije nastavka rada.

4. Provjeriti je li sve postavljeno korištenjem primjera, npr. od [ovuda](#).

- Pokrenuti ručno slike iz primjera prema uputama u `upute.txt`.
 - 5. Prema gornjim zahtjevima vježbe pripremiti slike za lab4
 - Za početak i pokretati ručno.
 - 6. Prema uputama složiti `compose.yaml` datoteku
 - Prvo instalirati dodatak `composer`, npr. sa:


```
sudo apt-get install docker-compose-v2
```
 - 7. Nekako izmjeriti vrijeme potrebno za ponovno pokretanje slike
 - Npr. u jednoj slici (njenom programu) ispisati trenutno vrijeme i završiti program (`exit(1)`), npr. na `SIGINT`), a na početku programa također ispisati vrijeme.
-

Korisni linkovi

- [Službene upute za Docker](#)
- [Docker CLI Cheat Sheet](#)
- [Coursera - Docker Cheat Sheet](#)
- [Docker Compose Quickstart](#)
- [Compose file reference](#)
- [HEALTHCHECK opcija u Dockerfile-u](#)
- [healthcheck opcija u .yaml datoteci](#)
- [restart opcija u .yaml datoteci](#)
- Složeniji primjer .yaml datoteke: [docker-compose-template.yaml](#)